

자기소개서

최성광

sungkwang0908@naver.com | 경력 3년 10개월 (의료정보시스템 운영·개발) | Microsoft AI School 10기 (2026.09 수료)

“레거시 의료정보시스템을 3년 10개월간 지켜온 운영 경험과, AI·클라우드 스택으로 서비스를 직접 세워본 구축 경험, 그리고 팀의 결과물을 검증해 돌아가게 만든 경험을 함께 가진 개발자입니다.”

1. 성장 과정 및 지원 동기 — 현실적인 선택에서 시작해, 스스로 확장해온 4년

저의 개발자 커리어는 거창한 사명감이 아니라 현실적인 판단에서 시작했습니다. 기계공학을 전공했지만 졸업을 앞두고 진로를 고민하던 시기, 전공 직무보다 개발 직무가 원하는 지역에서 더 빠르게 커리어를 시작할 수 있는 길이라고 판단해 코딩을 배우기 시작했습니다. 시작은 실용적인 선택이었지만, 그 선택을 지켜낸 것은 4년간의 태도였습니다.

에스파이온에서 3년 10개월간 CAMIS 의료정보시스템을 운영·개발하며 배운 것은 '기술보다 업무 흐름이 먼저'라는 사실입니다. 의료 개정 사양에 맞춰 비즈니스 로직을 수정하고, 기관마다 다른 업무 방식에 맞게 서비스를 커스터마이징했습니다. 신규 의료기관 오픈 현장에 직접 파견되어 의료진과 소통하며 데이터를 세팅하고 실시간으로 이슈에 대응했고, 2025년에는 일본 신규 병원 온보딩 출장까지 다녀왔습니다. 화면 하나의 오류가 곧 진료 지연으로 이어지는 환경에서, 저는 코드를 '동작하게 만드는 것'과 '안정적으로 운영되게 만드는 것'이 전혀 다른 문 제임을 체감했습니다.

그러나 3년 차에 접어들며 한 가지 위기감이 들었습니다. AI가 산업의 전제를 빠르게 바꾸고 있는데, 제가 다루는 기술은 온프레미스 레거시 환경에 머물러 있었습니다. AI와 클라우드를 제가 경험해보지 못한 영역이었고, 이대로라면 '유지보수는 잘하지만 새로운 것은 못 만드는 개발자'로 남을 것 같았습니다.

그래서 다음 단계를 먼저 준비한 뒤 움직였습니다. 퇴사를 준비하던 시기에 Microsoft AI School 10기에 지원해 면접을 거쳐 합격했고, 2026년 3월 퇴사와 동시에 과정에 참여했습니다. 공백 없이 곧바로 다음 단계로 넘어갈 수 있도록 미리 계획한 결정이었고, 남이 시켜서가 아니라 제 커리어의 방향을 스스로 판단해 선택한 전환이었습니다.

그 6개월 동안 세 번의 팀 프로젝트를 거쳤고, 마지막 프로젝트에서는 팀이 최우수상을 받았습니다. 이제 저는 레거시 시스템을 안정적으로 지켜온 운영 경험과, 새로운 기술 스택으로 서비스를 직접 세워본 구축 경험을 모두 가진 개발자로서 다음 자리를 찾고 있습니다.

2. 직무 역량 — 운영에서 얻은 신중함, 프로젝트에서 얻은 구축력

저의 강점은 '검증된 운영 감각'과 '스스로 확장하는 학습력'이 함께 있다는 점입니다.

실무에서는 3년 10개월간 ASP.NET(C#), Oracle PL/SQL, DevExpress, Vue.js 기반의 의료정보시스템을 담당했습니다. 서버 로그와 배치 서비스를 모니터링해 오류를 사전에 탐지하고, 장애 발생 시 원인을 신속히 분석해 대응했으며, 운영 안정성을 고려한 배포 관리를 수행했습니다. 여러 기관의 데이터를 처리하기 위해 DB Link를 활용하고, 연도·월별 통계 화면을 개발해 기관 운영진의 의사결정 데이터를 제공했습니다. 이 과정에서 '내 코드가 지금 돌아가는 서비스에 어떤 영향을 주는가'를 먼저 생각하는 습관이 몸에 붙었습니다.

AI School 프로젝트에서는 그 감각 위에 새로운 스택을 얹었습니다. 1차 프로젝트 '이약머약'에서는 프론트엔드 전체를 담당했지만, 서비스를 End-to-End로 직접 구축해보고 싶다는 생각을 팀에 제안해 서버 구축과 데이터 관리까지 함께 맡았습니다. Azure VM 위에 Nginx와 FastAPI, PostgreSQL로 서버 환경을 직접 세우고, 의약품 데이터를 크롤링·정제해 학습 데이터셋 3,773장을 구축했으며, Azure Custom Vision 모델을 연동해 실제 도메인으로 서비스를 공개했습니다.

2차 프로젝트 '채워줘, 홈즈!'에서는 백엔드 서버와 핵심 AI 파이프라인을 전담했습니다. 빈방 사진 한 장에서 카메라 파라미터를 복원해 미터 단위 좌표계를 세우고, Gemini Robotics-ER로 가구 배치를 추론한 뒤 nvdiffrast로 GPU 렌더링해 원본 사진에 합성하는 4단계 파이프라인을 설계·구현했습니다. 특히 LLM이 임의로 위치를 정하지 않도록 좌표 규칙을 계산 가능한 수식 형태로 프롬프트에 명시해 출력을 안정화했고, 무거운 모델들을 앱 수명주기에 바인딩해 요청당 지연을 최소화했습니다. HTTPS 프론트가 HTTP 백엔드를 호출할 수 없는 문제도 VM에 Caddy를 직접 설치해 해결했습니다.

3차 프로젝트 'ForeShield'에서는 대화 도메인 백엔드와 DB를 맡았습니다. 하나의 기능을 여러 사람이 나눠 만든 뒤 합치는 구조여서, 제 코드만 맞게 짜는 것으로는 서비스가 돌아가지 않았습니다. 대화 저장과 실시간 응답 전달(SSE), 인증 검증, 데이터 보존·삭제 처리를 구현했고, Alembic으로 DB 스키마 변경 이력을 관리했습니다. GitHub Projects로 작업을 이슈 단위로 관리하고 PR마다 CI가 검증하는 환경에서 개발했습니다.

익숙한 언어와 프레임워크가 아니어도, 문제를 구조로 나눠 하나씩 검증하며 결과물까지 만들어낼 수 있다는 것을 세 번의 프로젝트로 증명했다고 생각합니다.

3. 문제 해결 경험 — 지표는 좋은데 실전에서 틀리는 모델을 데이터로 고치다

'이약머약'에서 가장 어려웠던 문제는 학습 지표와 실제 성능의 괴리였습니다.

공공 데이터로 학습한 알약 분류 모델은 학습 지표가 매우 높게 나왔지만, 실제로 휴대폰으로 촬영한 사진에서는

인식률이 크게 떨어졌습니다. 원인을 추적해보니 원본 데이터의 배경이 단 4종뿐이라, 모델이 알약의 형태와 각인이 아니라 배경 패턴을 학습한 과적합 상태였습니다.

저는 모델 파라미터를 조정하는 대신 데이터를 바꾸는 쪽을 선택했습니다. 사용자가 실제로 알약을 놓는 환경 — 탁자, 손바닥 등 — 의 배경 이미지를 직접 수집하고, rembg로 알약 배경을 제거한 뒤 수집한 배경과 합성하는 Background Augmentation 파이프라인을 구현했습니다. 이때 회전 증강도 함께 실험했는데, $\pm 45^\circ$ 로 회전시키자 알약의 각인이 훼손되고 객체가 프레임을 벗어나는 문제가 발생했습니다. 여러 범위를 시도한 끝에 $0\sim 10^\circ$ 의 미세 회전만 적용하는 것이 왜곡 없이 다양성을 확보하는 최적점이라는 결론을 얻었고, 노이즈와 가림 같은 과도한 증강은 배제했습니다.

재학습 후에도 이지엔6프로, 지르텍정 등 일부 알약에서 인식 실패가 남아 있었습니다. 전체를 다시 손보는 대신 재현율이 기준치 이하인 취약 클래스 7종만 선별해 해당 객체의 특징을 강조한 맞춤형 추가 증강을 적용했고, 클래스 간 성능 격차를 크게 줄였습니다.

'채워줘, 홈즈!'에서도 같은 방식으로 접근했습니다. 처음에는 Stable Diffusion으로 가구를 합성했지만 방 구조가 변형되고 가구가 왜곡되어 원본 공간이 보존되지 않았습니다. 프롬프트를 수십 번 수정해도 생성 모델의 특성상 한계가 있다고 판단해, 접근 자체를 GLB 3D 모델 기반 실측 렌더링으로 전환했습니다. 카메라 파라미터를 복원해 좌표계를 세우고 GPU 래스터라이저로 가구만 그려 원본 사진 위에 덮는 방식이었고, 결과적으로 배경을 100% 보존하면서 실제 치수에 맞는 배치를 얻었습니다.

두 경험에서 저는 파라미터를 만지기 전에 원인을 규명하고, 필요하면 접근 방식 자체를 바꾸는 것이 더 빠른 길일 수 있다는 것을 배웠습니다.

4. 협업 경험 — 대립은 조율하고, 일정은 지키는 사람

저는 협업에서 두 가지를 중요하게 생각합니다. 기술적 의견 차이는 근거로 좁히고, 일정에 관한 것은 우선순위로 정리하는 것입니다.

'이약머약'에서는 백엔드의 데이터 조회 방식을 두고 팀원과 의견이 갈렸습니다. 저는 SQL을 XML Mapper에 명시적으로 관리하는 방식을, 다른 팀원은 ORM 방식을 주장했습니다. 저는 복잡한 조회 쿼리를 직접 제어하고 튜닝할 수 있어야 한다고 봤고, 팀원은 개발 속도와 코드 일관성을 중시했습니다. 어느 한쪽이 틀린 주장이 아니었기에, 서로의 장단점을 정리해 공유한 뒤 '두 방식을 배제할 필요가 없다'는 결론에 도달했습니다. 결과적으로 단순 CRUD는 SQLAlchemy ORM으로, 복잡한 조회는 SQL Mapper로 처리하는 구조를 함께 채택했고, 각자가 근거로 제시한 장점을 모두 살릴 수 있었습니다. 이 경험에서 기술적 대립은 승패를 가리는 문제가 아니라 적용 범위를 나누는 문제일 수 있다는 것을 배웠습니다.

'채워줘, 홈즈!'에서는 반대로 분명하게 선을 그어야 하는 상황이 있었습니다. 저는 핵심 파이프라인 개발과 전체 프로젝트 관리를 맡고 있었는데, 파이프라인 구현에 필요한 시간 자체가 빠듯한 상황이었습니다. 그런 시점에 팀원이 새로운 기능을 제안했고, 아이디어 자체는 좋았지만 지금 착수하면 핵심 기능조차 완성하지 못할 위험이 컸습니다.

그래서 이번에는 조금 더 단호하게 의견을 냈습니다. 남은 일정과 파이프라인 각 단계의 작업량을 함께 확인하며 '먼저 동작하는 프로토타입을 완성하고, 시간이 남으면 제안한 기능을 붙이자'고 제안했고, 팀원도 수긍해 우선순위를 합의했습니다. 결과적으로 4단계 파이프라인을 기한 내에 완성했고, 확보된 시간에 3D 뷰어와 음성 안내(TTS) 같은 확장 기능까지 붙일 수 있었습니다.

'ForeShield'에서는 협업의 방식이 달랐습니다. 각자 기능을 나눠 만들다 보니 개별 기능은 동작해도 합쳐놓으면 연결 부분에서 문제가 드러났습니다. 저는 서비스 QA를 맡아 제 담당 영역만이 아니라 화면과 서버 양쪽을 확인하고, 발견한 문제를 전달하며 반영 상태를 관리했습니다. 최종 결과보고서도 검토해 문서와 실제 서비스가 어긋난 부분을 바로잡았습니다.

세 사례 모두 제 담당 범위 안에서만 판단했다면 나오지 않았을 결과였습니다. 협업은 역할을 나누는 데서 시작하지만, 나눈 것을 다시 합치는 데서 완성된다고 생각합니다.

5. 입사 후 목표 — 지킬 수 있는 서비스를 만드는 개발자

저는 만드는 것과 지키는 것을 모두 할 수 있는 개발자가 되고 싶습니다.

3년 10개월간 운영 현장에 있으면서, 잘 만든 기능보다 잘 버티는 시스템이 사용자에게 더 큰 가치를 준다는 것을 배웠습니다. 동시에 AI School의 두 프로젝트를 통해 새로운 기술로 서비스를 세우는 일의 즐거움도 알게 되었습니다. 이 두 가지는 상충하지 않습니다. 오히려 운영을 아는 사람이 만들 때 더 오래 가는 서비스가 나온다고 믿습니다.

입사 후에는 먼저 담당 서비스의 도메인과 코드베이스를 빠르게 파악하는 데 집중하겠습니다. 신규 의료기관 온보딩 때마다 각 기관의 업무 흐름을 이해하는 것에서 출발했던 방식이 새로운 조직에 적응하는 데도 유효할 것이라 생각합니다. 이후에는 장애 대응과 배포 관리에서 팀이 믿고 맡길 수 있는 사람이 되고, 중장기적으로는 프로젝트에서 다룬 클라우드 인프라와 AI API 연동 경험을 살려 기존 서비스에 새로운 기술을 안전하게 도입하는 역할을 하고 싶습니다.

레거시를 지킬 줄 알고 새로운 것을 만들 줄도 아는 개발자로, 귀사의 서비스가 안정적으로 운영되면서도 기술적으로 도약하는 데 기여하겠습니다.